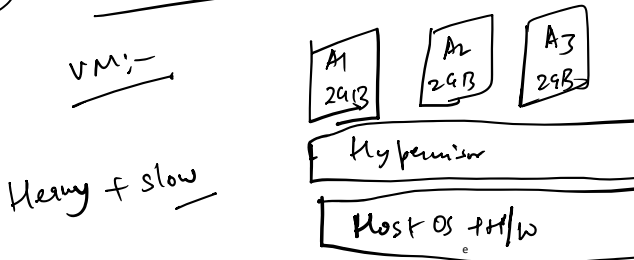
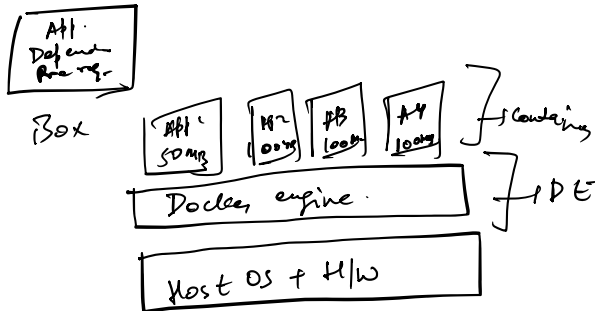


- ① VM, Container, Hypervisor
  - ② Docker
  - ③ K8S
  - ④ OpenShift
- } → Diff.

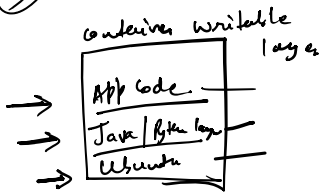
## ① VM, Container, Hypervisor:-



## Container:-



- ① Namespace — Process → IP, Network, Mount, etc.
- ② Cgroups — How much CPU, Disk, IO given by the process.
- ③ Union filesystem —



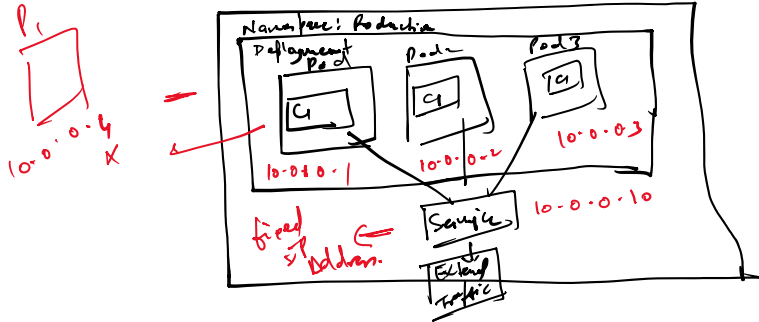
Docker — Make the container run.

- ① Dockerfile
- ② Image → Snapshot of App + Dependencies
- ③ Registry → Store the image.
- ④ Container → Run instance from Image
- ⑤ Docker Engine → Docker run container on a host. → Docker file.

```
FROM node:18-alpine # base layer (existing image)
WORKDIR /app
COPY package*.json ./ # cached unless deps change
RUN npm ci --only=production # install deps as a layer
COPY . . # copy app source
EXPOSE 3000
CMD ["node", "server.js"]
```

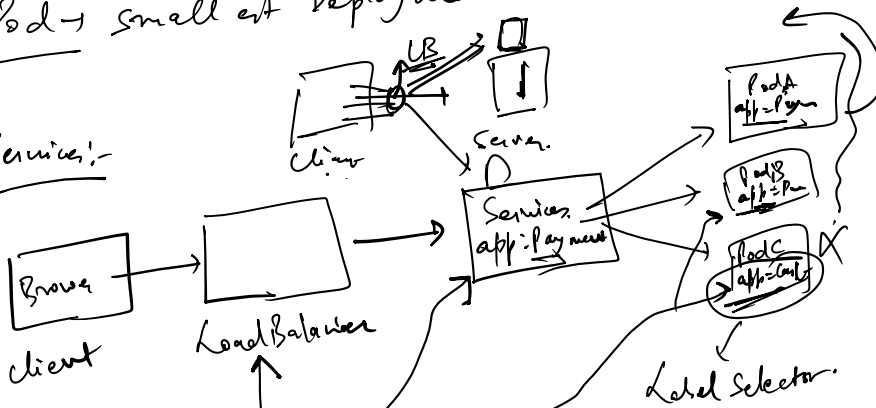
Docker file  $\rightarrow$  Image  $\rightarrow$  Registry  $\rightarrow$  Containers.

K8s (Kubernetes)  $\rightarrow$  K8s cluster.



Pod  $\rightarrow$  smallest Deployable Unit.

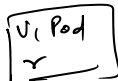
Services:-



```
apiVersion: v1
kind: Service
metadata:
  name: payment-service
spec:
  type: LoadBalancer
  selector:
    app: payment # matches pods with this label
  ports:
    - port: 443
      targetPort: 3000
```

Deployment:- Scalability, Auto healing.

Step 1



Step 2



Step 3

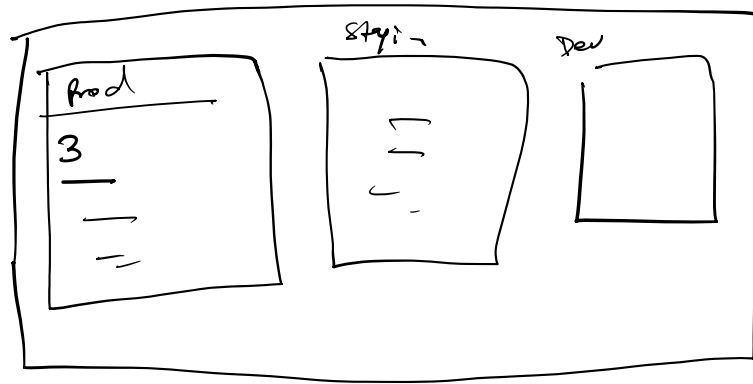


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: restaurant-service
spec:
```

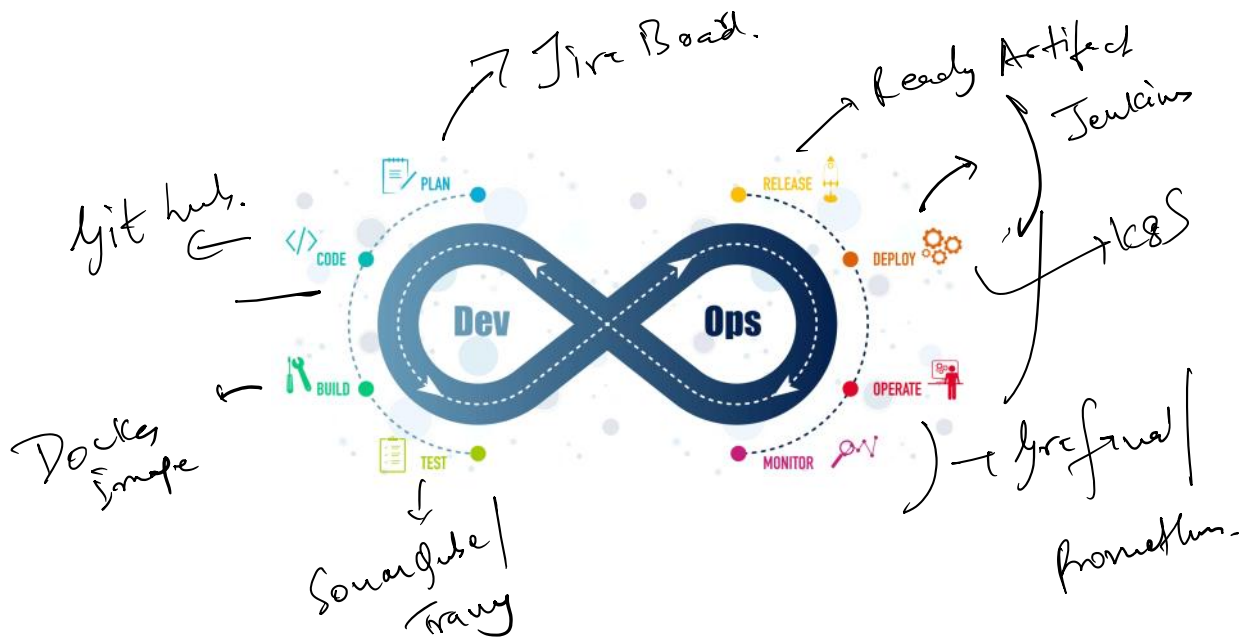
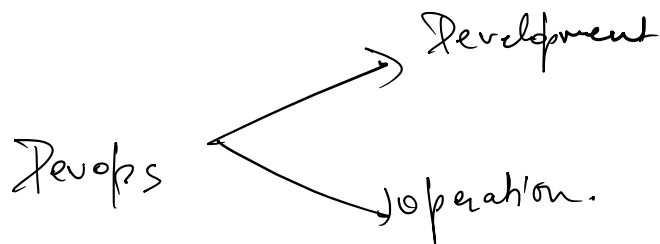
```

replicas: 3
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
selector:
  matchLabels:
    app: restaurant
template:
  metadata:
    labels:
      app: restaurant
spec:
  containers:
    - name: api
      image: restaurant-api:v2
      ports:
        - containerPort: 8080
      readinessProbe:
        # must pass before traffic is sent
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 5

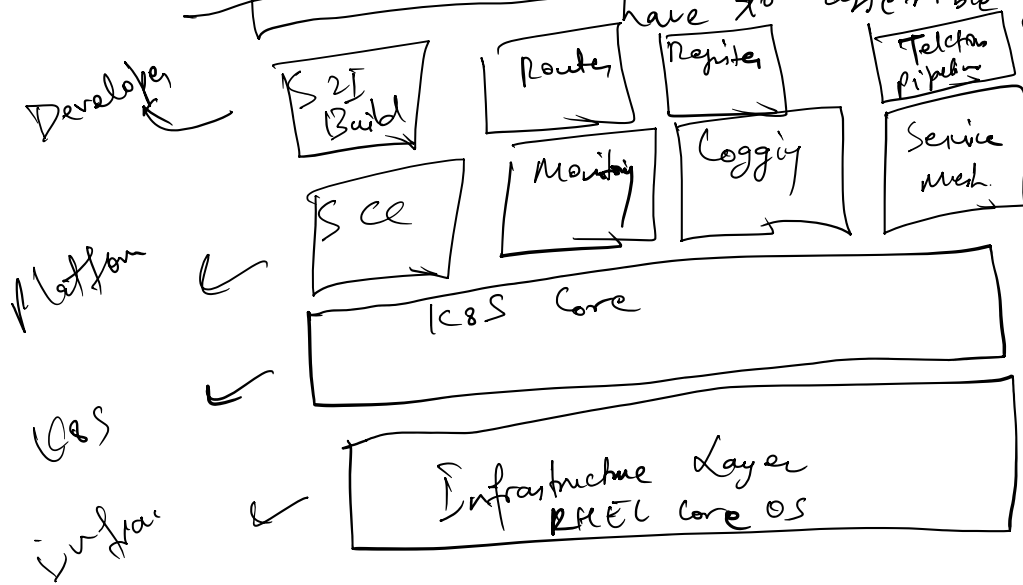
```



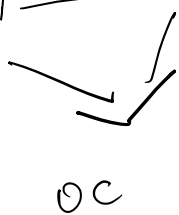
OpenShift:-



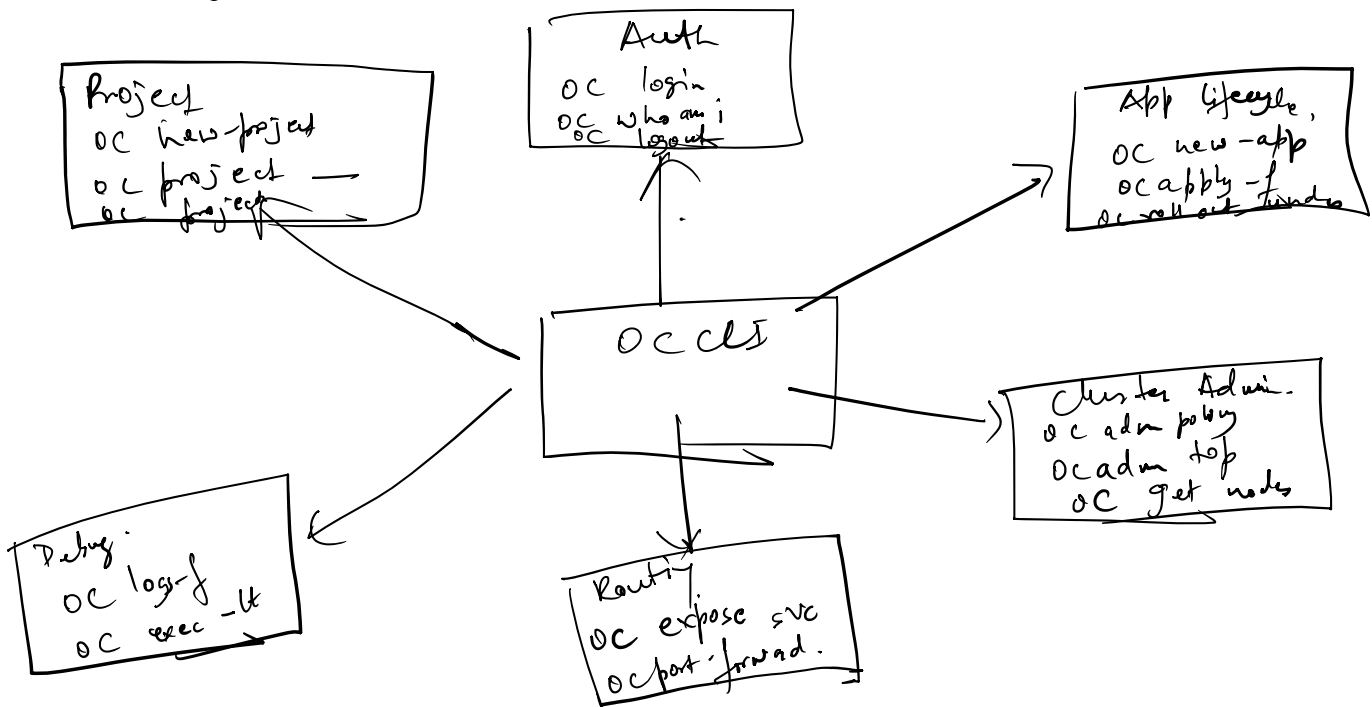
Openshift → K8S + everything else you do  
 GUI have to assemble yourself.

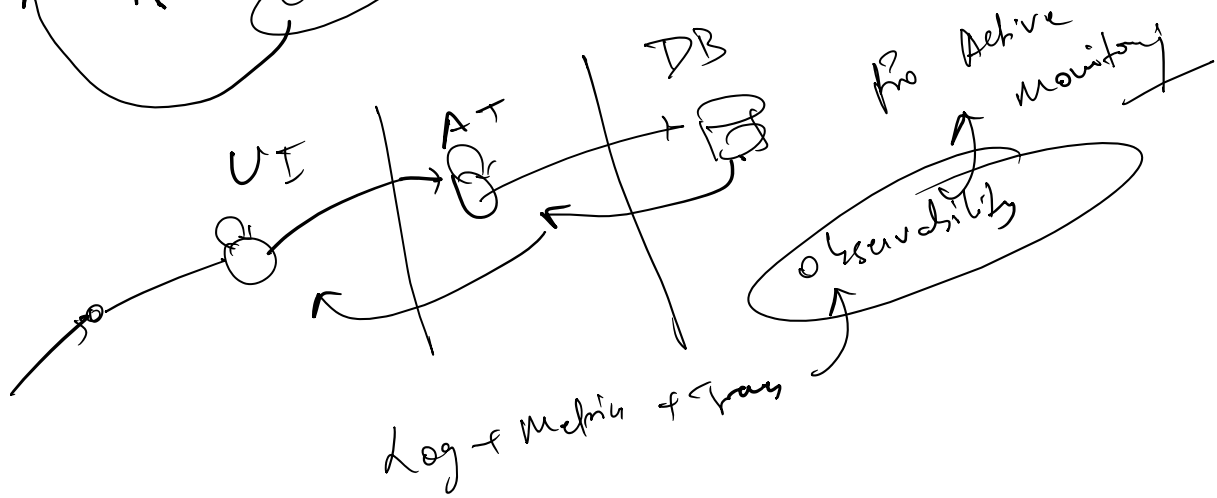
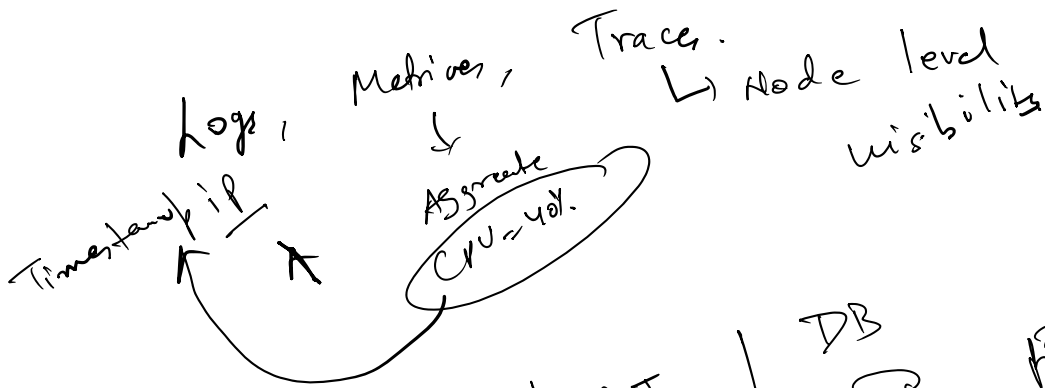
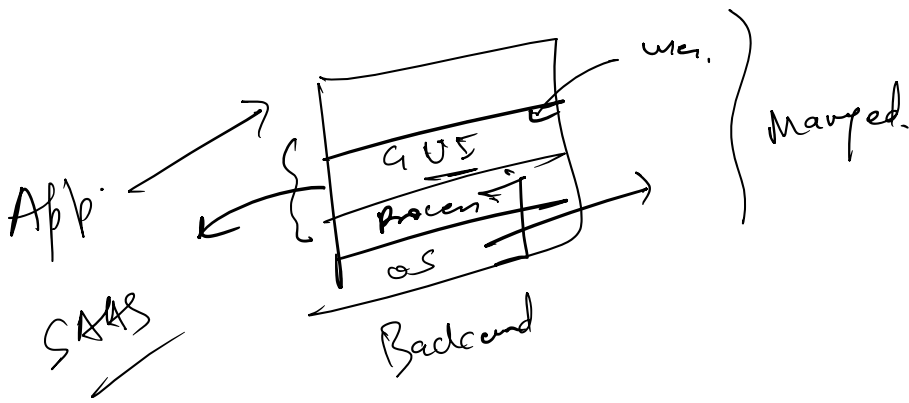


Openshift CLI (oc) & Web console



K8S → kubectl  
 Openshift → OC

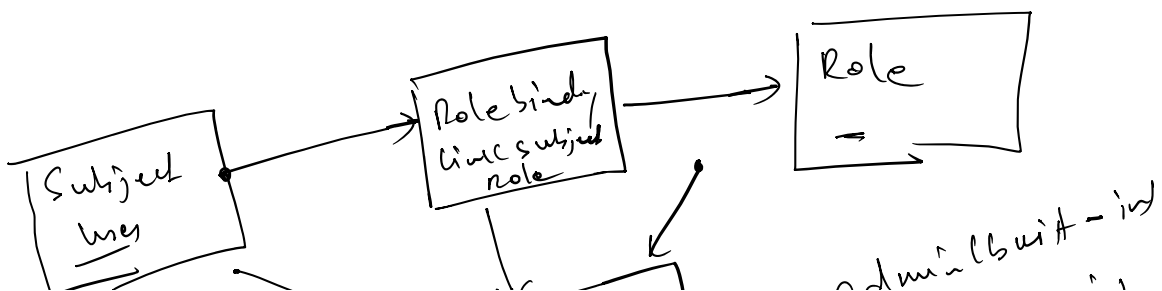


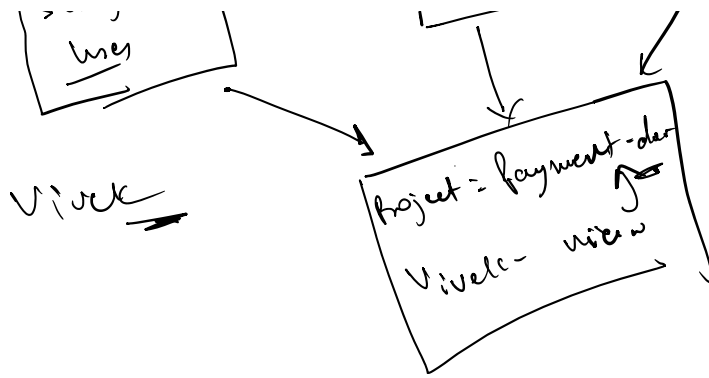


## ② RBAC Basics

Role Based Access Control.

Role binding =



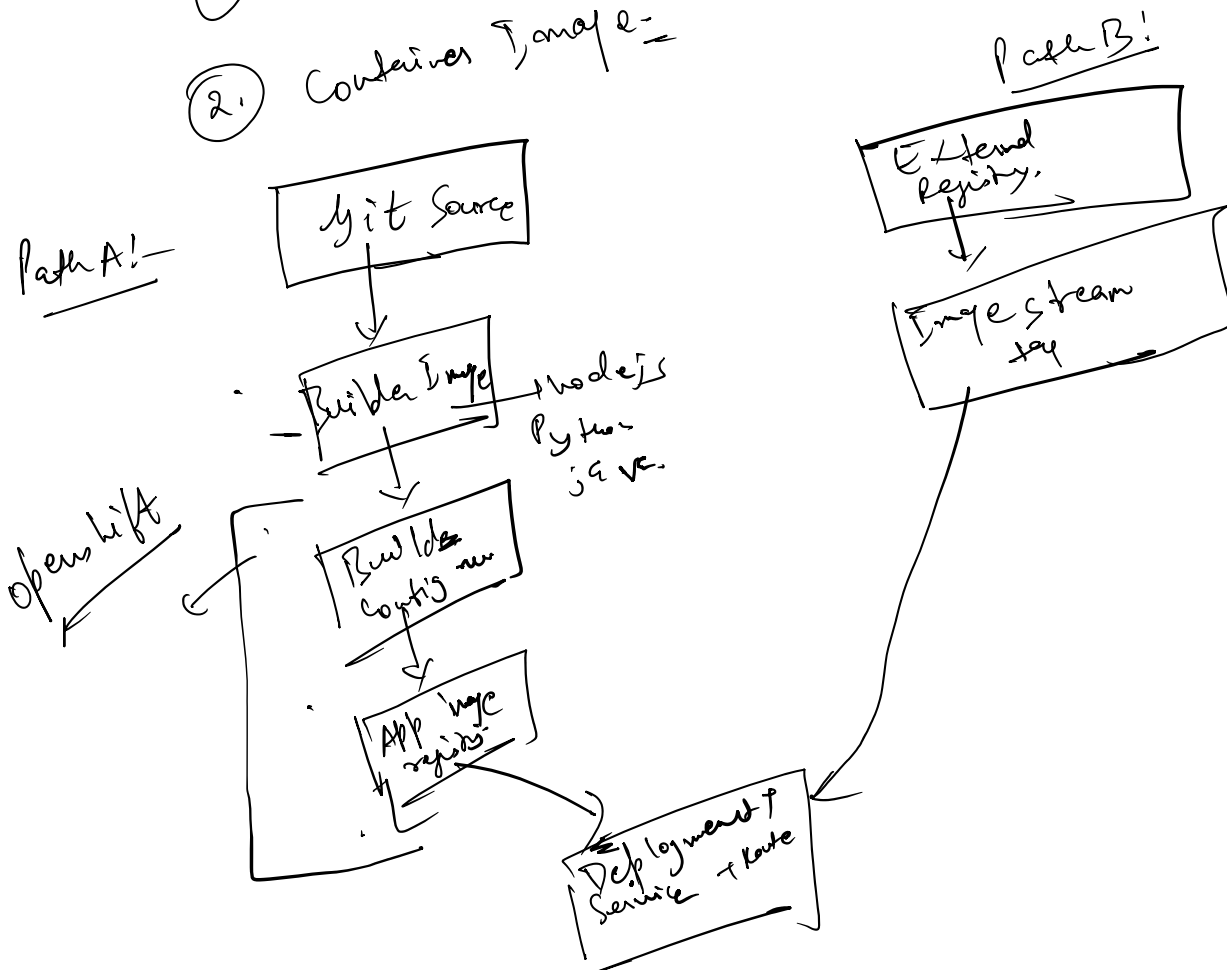


admin built-in  
edit built-in  
view built-in

\* Deploy app via:-

① S2I

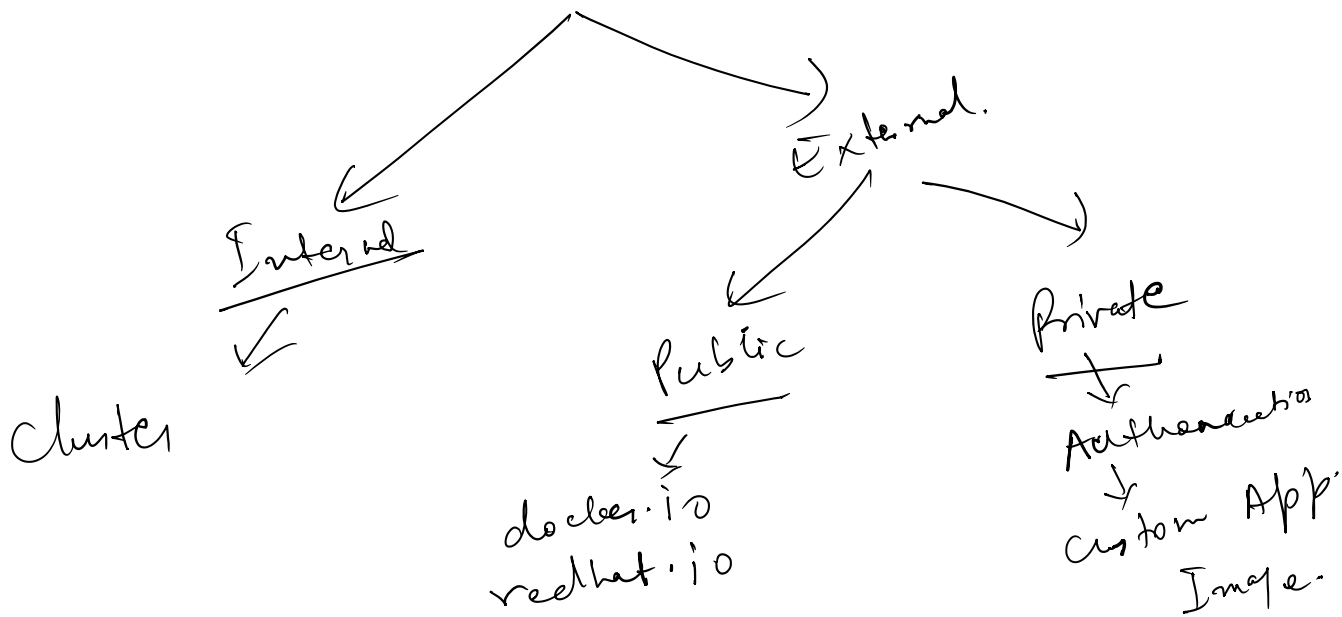
② Containers Image:-



Path A:-

Source to Image

Registry → store Image



Path-B :- Already Image → docker.io → Public Registry.

